

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67130 – מבוא למדעי המחשב בדגש על תכנות פרוצדורלי

מועד א' - סמסטר א' - תשס"ז – 2006-2007

המורה: פרופ' ג'ף רוזנשיין

משך הבחינה: שלוש שעות

ת.ז.

מס' מחברת:

9.2.07

תאריך:

הנחיות:

1. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק'). במידה ויוגשו פתרונות ליותר משלוש שאלות, הציון יקבע על פי שלוש השאלות הראשונות בלבד.
2. יש לכתוב פתרון לכל אחת מהשאלות שבחרת **במחברת נפרדת**. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
3. במידה ובחרתם לענות על השאלות האמריקאיות עליכם לסמן את תשובתכם על גבי טופס המבחן עצמו ולהגיש אותו יחד עם מחברות הבחינה.
4. יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת וגם על גבי טופס הבחינה עצמו.
5. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
6. יש למחוק טיוטות באופן ברור.
7. יש להשאיר שוליים.
8. אסור להשתמש בעט אדום.
9. הקפידו על סגנון תכנותי נאות: כתבו באופן מודולארי, אל תכפילו קוד וכו'. מותר להוסיף פונקציות עזר מעבר לפונקציות הנדרשות בשאלה.
10. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
11. פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
12. אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר תמיד להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בקלט (למשל מותר להקצות מערך בן תא אחד פעם אחת).
13. בכל השאלות מותר להניח שהקלט חוקי (ומקיים את תנאי השאלה).

שאלה 1: שאלות אמריקאיות (33 נק').

ענו על 8 מתוך 10 הסעיפים הבאים בלבד. סמנו באופן ברור את שני הסעיפים עליהם בחרתם לא לענות. עבור כל שאלה, בחרו את התשובה הנכונה ביותר. סמנו תשובה אחת בלבד! ערכו של כל סעיף 4 נקודות (נק' אחת מקבלים חינם). סמנו בבירור את תשובותיכם על גבי טופס המבחן והגישו אותו. ודאו כי רשמתם את מס' תעודת הזהות שלכם ואת מספר המתברת בראש טופס הבחינה!

```
f(int n){
    for (int i=0; i<n; i++){
        j = n;
        while (j>0){
            j = j/3;
        }
    }
}
```

א. מהו החסם העליון ההדוק ביותר על זמן הריצה של הפונקציה $f()$ על קלט n ?

- a. $O(n)$
- b. $O(n^3)$
- c. $O(n\sqrt[3]{n})$
- d. $O(n \log_3 n)$

```
public class Value {
    public int i;
    public Value() {i=15;}
}

public class Test {
    public static void foo1() {
        int i = 5;
        Value v = new Value();
        v.i = 10;
        foo2(v,i);
        System.out.print(v.i);
        System.out.print(i);
    }

    public static void foo2(Value v,
                           int i) {
        v.i = 30;
        i = 10;
        Value val = new Value();
        v = val;
        System.out.print(v.i);
    }
}
```

ב. מה יודפס (משמאל לימין) כאשר נריץ את השיטה `foo1`?

- a. 15 30 5
- b. 5 10 30
- c. 5 30 30
- d. 15 10 30
- e. 15 5 10

```
class A{}
class B extends A{
    public B(){
        System.out.println("B constructor");
    }
}
```

ג. נתונות המחלקות הבאות:

איזו מהטענות הבאות אינה נכונה:

- a. הקונסטרוקטור של B יקרא לקונסטרוקטור הדיפולטיבי של A.
- b. הקונסטרוקטור הדיפולטיבי של Object לא יקרא כי לא A אין קונסטרוקטור מפורש שיקרא לו.
- c. הקונסטרוקטור הדיפולטיבי של Object יקרא לאחר הדפסת המחרוזת "B constructor".
- d. קודם יקרא הקונסטרוקטור הדיפולטיבי של A ואז תודפס המחרוזת "B constructor".
- e. בחר בסעיף זה אם הטענות b ו- c אינן נכונות.

ד. איזו מהטענות הבאות אינה נכונה?

- a. אי אפשר לעשות override לשיטה סטטית.
- b. לשיטה שעושה override מותר לשנות את הרשאות הגישה רק בתנאי שהיא מגדירה הרשאות מגבילות פחות.
- c. מספר, סוג וסדר הארגומנטים של שיטה העושה override חייב להיות זהה לאלו בשיטה המקורית.
- d. אי אפשר לעשות override לשיטה שהוגדרה כ final.
- e. בחר בסעיף זה אם כל הטענות נכונות.

ה. רוצים למיין בסדר יורד את המערך הבא (נתון משמאל לימין): 25 56 18 10 12 90 15
עבור איזה מאלגוריתמי המיין הבאים, כפי שנלמדו בכיתה, נקבל כשלב ביניים את המערך:
90 56 25 10 12 18 15

- a. מיון הכנסה (insertion sort).
- b. מיון בחירה (selection sort).
- c. מיון בועות (bubble sort).
- d. מיון מהיר (quick-sort) כאשר pivot נבחר כאיבר הראשון.
- e. אף אחד מהם.

ו. מהו זמן הריצה של חיפוש ריבועי (quadratic search) במערך ממויין בגודל n במקרה הגרוע ביותר?

- a. $O(n^2)$
- b. $O(n)$
- c. $O(\sqrt{n})$
- d. $O(\log n)$

ז. נתונה המחלקה שמשמאל.

עבור אילו מהרשימות הבאות תחזיר המתודה what() את הערך true (בהינתן n מטיפוס IntNode).

- a. n->4->4->3->2->null
- b. n->4->3->2->3->null
- c. n->1->2->3->4->null
- d. תשובות a ו- c נכונות

```
class IntNode{
    private int _data;
    private IntNode _next;
    public IntNode(int data){
        _data = data;
    }
    public static boolean what(
        IntNode node){
        IntNode t = node;
        IntNode h = node._next;
        boolean ok = false;
        if (h!=null)
            ok = (t._data<h._data);
        while (h!=null) {
            if (ok!=(t._data<h._data))
                return false;
            t = h;
            h = h._next;
        }
        return true;
    }
}
```

ח. מה נוכל להגיד בוודאות על חלק התוכנית הבא, שרץ ללא שגיאה (אין להניח שנעשה שימוש ב-naming conventions כלשהם):

```
plane.rise(meter);
```

- a. rise חייב להיות שיטה.
- b. meter חייב להיות משתנה פשוט (primitive).
- c. rise חייב להיות שדה.
- d. plane הוא בהכרח שם של מחלקה.
- e. plane הוא בהכרח אובייקט.

ט. בהתייחס לקטע הקוד שמשמאל, איזה מקטע קוד לא יגרום לשגיאה מסוג כלשהו (בזמן ריצה או בזמן קומפילציה)?

```
class Base {...}
```

```
interface Inter {...}
```

```
class Derived extends Base
    implements Inter{...}
```

- a. `Inter inter = new Base();`
- b. `Base base = new Inter();`
- c. `Inter inter = new Derived();`
- d. `Base base = (Base)inter;`
- e. `Base base = new Base();`
- e. `Inter inter = (Inter)base;` בחר בסעיף זה אם כל המקטעים יכולים לגרום לשגיאה.

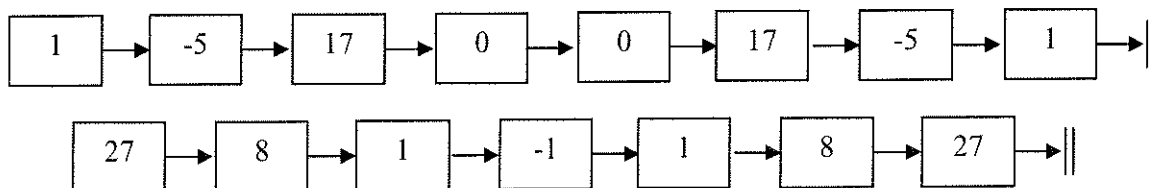
```
static int foo(int a, int b) {
    if (a==0)
        return b;
    return foo(a/b,b*3);
}
```

- י. מה תחזיר השיטה `foo(333,3)`?
- a. 111
 - b. 3
 - c. 81
 - d. 33
 - e. 27

שאלה 2: זיהוי פאלינדרום ברשימה מקושרת (33 נק').

הגדרת פאלינדרום: נאמר שרשימה מקושרת המחזיקה מספרים מטיפוס `int` היא פאלינדרום אם המספרים המופיעים ב-nodes יוצרים סדרה שקריאתה משמאל לימין ומימין לשמאל תיתן תוצאה זהה.

דוגמא: כל אחת מהרשימות הבאות היא פאלינדרום:



לרשותכם עומדת מחלקה בשם `Node` אותה כבר מימשנו עבורכם כך שאין צורך שתמשו אותה.

במחלקה `Node` מימשנו את שתי השיטות הבאות:

```
public Node getNext()
```

שיטה זו מחזירה את ה-`Node` הבא ברשימה או `null` אם אין כזה.

```
public int getData()
```

שיטה זו מחזירה את המספר המוכל ב-`node` הנוכחי.

סעיף א' (17 נק'):

כתבו שיטה (לא רקורסיבית, סטטית, השייכת למחלקה ללא `data members`) בשם `isPalindrome`, המקבלת פרמטר יחיד מסוג `Node` המייצג ראש של רשימה מקושרת ומחזירה ערך בוליאני של אמת אם ורק אם הרשימה שניתנה היא פאלינדרום. חתימת השיטה היא:

```
public static boolean isPalindrome(Node head)
```

הניחו כי ה- Node שמועבר לשיטה אינו null, ושהרשימה בנויה בצורה חוקית (אינה מכילה מעגלים ומסתיימת עם מצביע ל-null).

במהלך הפתרון של סעיף זה עליכם להשתמש רק בכמות קבועה של זיכרון נוסף (שאינה תלויה באורך הרשימה שקבלתם), אסור להשתמש ברקורסיה, ואין לשנות את מבנה הרשימה המקושרת או את תוכנה.

על הפתרון שלכם לעבוד בסיבוכיות של $O(n^2)$. הראו כי הפתרון שנתתם מקיים דרישה זו (3 נק' מתוך 17).

סעיף ב' (16 נק'):

פתרו שוב את אותה הבעיה של זיהוי רשימה שהיא פאלינדרום, הפעם בסיבוכיות זמן ריצה של $O(n)$. לצורך פתרון סעיף זה, מותר לשנות את מבנה הרשימה. כלומר מותר לשנות את המצביעים לאיבר הבא ברשימה (אך אסור להשתמש ברקורסיה ואין להקצות זיכרון נוסף מעבר לכמות קבועה של זיכרון). לצורך שינוי הרשימה, הניחו כי ב- Node מימשנו כבר עבורכם השיטה הבאה:

```
שיטה זו משנה את המצביע ב- Node כך שיצביע ל- next. public void setNext(Node next)
```

בשני המקרים, הניחו שהרשימה שניתנה לכם אינה null ושהיא בנויה באופן חוקי (אינה מכילה מעגלים, מסתיימת במצביע ל-null). הסבירו את פתרונכם והראו כי הוא עובד בסיבוכיות הדרושה (3 נק' מתוך 16).

שאלה 3: רקורסיה (33 נק').

עזריאל רוצה לבנות מגדלי קוביות בעלי גובה אחיד height. לרשותו קוביות בגדלים שונים (גודל כל קובייה הוא מספר חיובי שלם). גדלי הקוביות נתונים במערך blocks מטיפוס `int[]`. עזריאל רוצה לחלק את הקוביות למגדלים כדי לבנות מספר מקסימלי של מגדלים, כל מגדל בגובה height בדיוק. כל קובייה יכולה כמובן לשמש לבנייתו של מגדל אחד בלבד, ויתכן שיוותרו קוביות מיותרות בסוף הבנייה.

לדוגמא: בהינתן סט הקוביות: `blocks={5,2,6,6,1,1,4}` וגובה מגדל רצוי `height = 7` ניתן לבנות מגדלים ע"י החלוקה `{4+2+1}`, `{6+1}` כאשר נותרו הקוביות בגודל 5,6. אך חלוקה אחרת נותנת יותר מגדלים: `{6+1}`, `{6+1}`, `{2+5}` כאשר נותרת קובייה בגודל 4.

היות ועזריאל מעוניין במספר המקסימלי של המגדלים שאפשר לבנות, יש למצוא עבורו את החלוקה השנייה שבדוגמא (או כל חלוקה אחרת שבונה מספר זהה של מגדלים).

ממשו שיטה `buildTowers` שמקבלת מערך `blocks` המכיל את גדלי הקוביות, ואת גובה המגדלים הרצוי - `height` ומוצאת חלוקת קוביות (אחת בלבד) שנותנת מספר מגדלים מקסימלי.

עליכם לפתור את הבעיה באמצעות רקורסיה. השיטה הרקורסיבית לא חייבת להיות `buildTowers` עצמה. ניתן להגדיר שיטות עזר בעלות חתימה שונה שתעשנה את העבודה ברקורסיה.

על השיטה להדפיס בשורה נפרדת את גדלי הקוביות שיש להציב בכל מגדל. אין בהדפסה חשיבות לסדר של המגדלים או לסדר הדפסת גדלי הקוביות בתוך כל מגדל. ערך ההחזרה של השיטה צריך להיות מספר המגדלים שניתן לבנות.

```
שיטה חתימה כדלקמן: public int buildTowers(int[] blocks, int height)
```

הניחו כי המערך `blocks` אינו null, וכי כל גדלי הקוביות הנתונים בו הם אכן חיוביים. בעת הפתרון מותר להקצות סה"כ עד $O(n)$ זכרון כאשר `n` הוא גודל המערך `blocks` הנתון. עם זאת, מותר להגדיר מחלקות חדשות במידת הצורך. שימו לב שהשיטה `buildTowers` אינה סטטית וניתן להשתמש בשדות המחלקה בה היא נמצאת במידת הצורך.

שאלה 4: Shell Sort (33 נק').

Shell Sort הוא אלגוריתם מיון שמבוסס על וריאציה של Insertion Sort. מחולק לשלבים, כשבכל שלב מתבצעת סדרת מיון Insertion Sort במערך על תאים הנמצאים במרחק k זה מזה. בכל שלב בריצת האלגוריתם, מתבצעים מיונים עבור מרווח k קטן והולך, עד שלבסוף, בשלב האחרון של האלגוריתם, מתבצע Insertion Sort רגיל עבור מרווח $k=1$.

לדוגמא, בהינתן המערך הבא המכיל 10 מספרים שלמים ואינו ממוין:

32	95	16	82	24	66	35	19	75	54
----	----	----	----	----	----	----	----	----	----

עבור $k=4$ יתבצעו מיונים עבור האיברים המופיעים במערך בקפיצות של 4 תאים. כלומר, המיון הראשון שיתבצע יהיה עבור התאים במיקום 0,4,8 בלבד וימין רק אותם בסדר עולה. המיון השני ימין רק את התאים 1,5,9. המיון השלישי ימין את התאים 2,6 בלבד, והמיון הרביעי יתבצע רק על תאים 3,7. עבור $k=4$ המערך ההתחלתי הוא:

32	95	16	82	24	66	35	19	75	54
----	----	----	----	----	----	----	----	----	----

לאחר ביצוע המיון על תאים 0,4,8 יתקבל המערך:

24	95	16	82	32	66	35	19	75	54
----	----	----	----	----	----	----	----	----	----

לאחר מיון ה- Insertion Sort השני על התאים 1,5,9 יתקבל המערך:

24	54	16	82	32	66	35	19	75	95
----	----	----	----	----	----	----	----	----	----

וכן הלאה. לבסוף לאחר ביצוע כל המיונים בקפיצות של 4 תאים יתקבל המערך:

24	54	16	19	32	66	35	82	75	95
----	----	----	----	----	----	----	----	----	----

בשלב הבא יקבע ערך k נמוך יותר, כגון $k=2$ ובו יבוצעו כמה מיונים עבור הערכים הנמצאים במערך במרחק 2 צעדים זה מזה. עבור k כזה, המיון הראשון יתבצע על הערכים במקומות הזוגיים: 0,2,4,6,8 וימין רק אותם, ואילו המיון השני יתבצע על הערכים במקומות 1,3,5,7,9 בלבד. עבור $k=2$ המערך ההתחלתי:

24	54	16	19	32	66	35	82	75	95
----	----	----	----	----	----	----	----	----	----

ולאחר ביצוע שני המיונים יתקבל:

16	19	24	54	32	66	35	82	75	95
----	----	----	----	----	----	----	----	----	----

ב Shell Sort מתבצע לבסוף שלב מיון עבור $k=1$ בו מתבצע למעשה מיון Insertion Sort רגיל על כל המערך.

סעיף א' (28 נק').

`public static void shellSort(int[] data)`

כתוב שיטה סטטית:

שתקבל מערך לא ממוין של מספרים מטיפוס `int` ותבצע מיון Shell Sort על המערך, כך שלאחר המיון המספרים במערך יופיעו בסדר עולה (המספר הנמוך ביותר במערך יופיע בתא 0). הערך k צריך להתחיל כחצי מגודלו של המערך (מעוגל למטה) והוא צריך לקטון פי שתיים (שוב, מעוגל למטה) בכל שלב שבאלגוריתם, עד שבשלב האחרון מתבצע שימוש ב $k=1$.

סעיף ב' (5 נק').

מהו זמן הריצה של שיטת המיון שמימשתם במקרה הגרוע ביותר? מדוע ברוב המקרים ידרוש מיון Shell Sort פחות השוואות והחלפות של מספרים מאשר Insertion Sort על אף שבשלב האחרון במיון Shell Sort מתבצע למעשה מיון Insertion Sort רגיל?